

LAPORAN
“P13 STORED PROCEDURE, TRIGGER, DAN VIEW PADA
DATABASE SISTEM SIDANG TUGAS AKHIR“



Dosen Pengampu :
M.Ainurohman, S.Kom

Di Susun Oleh :

1. Asyroful Awlya Al Maysiry (2502007)

TEKNIK INFORMATIKA 1A D-III
POLITEKNIK PURBAYA

TAHUN 2025/2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini dibuat sebagai salah satu syarat untuk memenuhi tugas praktikum mata kuliah Aplikasi Basis Data mengenai penerapan View, Stored Procedure, Trigger, dan Query Laporan pada Database Sistem Sidang Tugas Akhir.

Dalam penyusunan laporan ini masih terdapat kekurangan, sehingga penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan selanjutnya. Semoga laporan ini dapat memberikan manfaat bagi pembaca.

Slawi, Juli 2026

Penulis

DAFTAR ISI

- Kata Pengantar
- Daftar Isi
- BAB I Pendahuluan
- BAB II Landasan Teori
- BAB III Langkah Praktikum
- BAB IV Hasil dan Pembahasan
- BAB V Penutup
- Daftar Pustaka
- Lampiran

BAB I PENDAHULUAN

1.1 Latar Belakang

Database merupakan komponen penting dalam pembangunan sebuah sistem informasi. Dengan menggunakan database, data dapat disimpan, dikelola, dan diolah secara terstruktur. Pada sistem sidang tugas akhir, pengelolaan data mahasiswa, dosen, ruangan, serta jadwal sidang harus dilakukan secara efektif agar tidak terjadi kesalahan dalam proses penjadwalan.

MySQL/MariaDB menyediakan beberapa fitur yang dapat mempermudah pengelolaan database, di antaranya View, Stored Procedure, dan Trigger. Ketiga fitur tersebut mampu meningkatkan efisiensi dalam pengolahan data serta mengurangi kesalahan yang dilakukan oleh pengguna.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

- Membuat View untuk menampilkan jadwal sidang.
- Membuat Stored Procedure untuk menambahkan data sidang.
- Membuat Trigger untuk mencatat aktivitas secara otomatis.
- Membuat Query Laporan pada database Sistem Sidang.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini yaitu mahasiswa mampu memahami penggunaan View, Stored Procedure, Trigger, serta Query Laporan dalam pengelolaan database menggunakan MySQL/MariaDB.

BAB II LANDASAN TEORI

2.1 Database

Database adalah kumpulan data yang saling berhubungan dan disimpan secara sistematis sehingga mudah diakses, dikelola, dan diperbarui.

2.2 View

View merupakan tabel virtual yang dihasilkan dari suatu perintah SELECT. View digunakan untuk mempermudah penyajian data tanpa mengubah data asli pada tabel.

2.3 Stored Procedure

Stored Procedure adalah kumpulan perintah SQL yang disimpan di dalam database dan dapat dijalankan kembali hanya dengan memanggil nama procedure tersebut.

2.4 Trigger

Trigger adalah program yang dijalankan secara otomatis ketika terjadi peristiwa INSERT, UPDATE, atau DELETE pada suatu table.

2.5 Query Laporan

Query laporan merupakan perintah SQL yang digunakan untuk menghasilkan informasi tertentu sesuai kebutuhan pengguna.

BAB III LANGKAH PRAKTIKUM

3.1 Membuat Database

Langkah pertama adalah membuat database dengan nama **db_sidang** kemudian memilih database tersebut agar siap digunakan.

Gambar 3.1 Pembuatan Database

```
MariaDB [(none)]> CREATE DATABASE db_sidang;
Query OK, 1 row affected (0.022 sec)

MariaDB [(none)]> USE db_sidang;
Database changed
MariaDB [db_sidang]> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| db_sidang  |
+-----+
1 row in set (0.000 sec)
```

3.2 Membuat Tabel

Selanjutnya dibuat enam tabel yaitu:

- mahasiswa
- dosen
- ruangan
- sidang
- penguji
- log_activity

Setiap tabel memiliki primary key dan foreign key sesuai hubungan antar tabel.

Gambar 3.2 Pembuatan Tabel

```
MariaDB [db_sidang]> CREATE TABLE mahasiswa (  
-> id_mahasiswa INT AUTO_INCREMENT PRIMARY KEY,  
-> nim VARCHAR(15),  
-> nama VARCHAR(100),  
-> prodi VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.118 sec)  
  
MariaDB [db_sidang]> CREATE TABLE dosen (  
-> id_dosen INT AUTO_INCREMENT PRIMARY KEY,  
-> nama_dosen VARCHAR(100),  
-> jabatan VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.026 sec)  
  
MariaDB [db_sidang]> CREATE TABLE ruangan (  
-> id_ruangan INT AUTO_INCREMENT PRIMARY KEY,  
-> nama_ruangan VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.018 sec)  
  
MariaDB [db_sidang]> CREATE TABLE sidang (  
-> id_sidang INT AUTO_INCREMENT PRIMARY KEY,  
-> id_mahasiswa INT,  
-> tanggal_sidang DATE,  
-> jam_mulai TIME,  
-> jam_selesai TIME,  
-> id_ruangan INT,  
-> FOREIGN KEY (id_mahasiswa)  
-> REFERENCES mahasiswa(id_mahasiswa),  
-> FOREIGN KEY (id_ruangan)  
-> REFERENCES ruangan(id_ruangan)  
-> );  
Query OK, 0 rows affected (0.042 sec)
```

```

MariaDB [db_sidang]> CREATE TABLE penguji (
  ->   id_penguji INT AUTO_INCREMENT PRIMARY KEY,
  ->   id_sidang INT,
  ->   id_dosen INT,
  ->   peran VARCHAR(50),
  ->   FOREIGN KEY (id_sidang)
  ->     REFERENCES sidang(id_sidang),
  ->   FOREIGN KEY (id_dosen)
  ->     REFERENCES dosen(id_dosen)
  -> );
Query OK, 0 rows affected (0.027 sec)

MariaDB [db_sidang]> CREATE TABLE log_activity (
  ->   id_log INT AUTO_INCREMENT PRIMARY KEY,
  ->   aktivitas VARCHAR(255),
  ->   waktu DATETIME
  -> );
Query OK, 0 rows affected (0.017 sec)

```

3.3 Input Data

Setelah tabel selesai dibuat kemudian dilakukan pengisian data mahasiswa, dosen, dan ruangan menggunakan perintah INSERT.

Gambar 3.3 Input Data

```

MariaDB [db_sidang]> INSERT INTO mahasiswa(nim,nama,prodi)
  -> VALUES
  -> ('2301001','Andi','Teknik Informatika'),
  -> ('2301002','Budi','Teknik Informatika'),
  -> ('2301003','Siti','Sistem Informasi');
Query OK, 3 rows affected (0.053 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [db_sidang]> INSERT INTO dosen(nama_dosen,jabatan)
  -> VALUES
  -> ('Dr. Ahmad','Ketua'),
  -> ('Budi Santoso','Anggota');
Query OK, 2 rows affected (0.039 sec)
Records: 2 Duplicates: 0 Warnings: 0

MariaDB [db_sidang]> INSERT INTO ruangan(nama_ruangan)
  -> VALUES
  -> ('Ruang A'),
  -> ('Ruang B');
Query OK, 2 rows affected (0.011 sec)
Records: 2 Duplicates: 0 Warnings: 0

```

3.4 Membuat View

View dibuat dengan nama **v_jadwal_sidang** yang digunakan untuk menampilkan jadwal sidang mahasiswa secara lengkap.

Gambar 3.4 Query View

```
MariaDB [db_sidang]> SHOW FULL TABLES
-> WHERE Table_type='VIEW';
Empty set (0.035 sec)

MariaDB [db_sidang]> DROP VIEW IF EXISTS v_jadwal_sidang;
Query OK, 0 rows affected, 1 warning (0.003 sec)

MariaDB [db_sidang]> CREATE VIEW v_jadwal_sidang AS
-> SELECT
-> m.nim,
-> m.nama,
-> m.prodi,
-> s.tanggal_sidang,
-> CONCAT(s.jam_mulai, ' - ', s.jam_selesai) AS jam_sidang,
-> r.nama_ruangan
-> FROM sidang s
-> JOIN mahasiswa m
-> ON s.id_mahasiswa=m.id_mahasiswa
-> JOIN ruangan r
-> ON s.id_ruangan=r.id_ruangan;
Query OK, 0 rows affected (0.040 sec)

MariaDB [db_sidang]> SELECT * FROM v_jadwal_sidang;
Empty set (0.025 sec)

MariaDB [db_sidang]>
MariaDB [db_sidang]> DROP PROCEDURE IF EXISTS sp_tambah_sidang;
Query OK, 0 rows affected, 1 warning (0.022 sec)
```

3.5 Membuat Stored Procedure

Stored Procedure dibuat dengan nama **sp_tambah_sidang** untuk menambahkan data sidang secara otomatis.

Gambar 3.5 Query Stored Procedure

```
MariaDB [db_sidang]> DELIMITER $$
MariaDB [db_sidang]>
MariaDB [db_sidang]> CREATE PROCEDURE sp_tambah_sidang(
  -> IN p_id_mahasiswa INT,
  -> IN p_tanggal DATE,
  -> IN p_jam_mulai TIME,
  -> IN p_jam_selesai TIME,
  -> IN p_id_ruangan INT
  -> )
  ->
  -> BEGIN
  ->
  -> INSERT INTO sidang(
  -> id_mahasiswa,
  -> tanggal_sidang,
  -> jam_mulai,
  -> jam_selesai,
  -> id_ruangan
  -> )
  ->
  -> VALUES(
  -> p_id_mahasiswa,
  -> p_tanggal,
  -> p_jam_mulai,
  -> p_jam_selesai,
  -> p_id_ruangan
  -> );
  ->
  -> END $$
Query OK, 0 rows affected (0.098 sec)

MariaDB [db_sidang]>
MariaDB [db_sidang]> DELIMITER ;
MariaDB [db_sidang]> CALL sp_tambah_sidang(
  -> 1,
  -> '2026-07-10',
  -> '08:00:00',
  -> '09:30:00',
  -> 2
  -> );
Query OK, 1 row affected (0.023 sec)
```


3.6 Membuat Trigger

Trigger dibuat dengan nama **trg_log_sidang** untuk mencatat aktivitas setiap penambahan data sidang.

Gambar 3.6 Query Trigger

```
MariaDB [db_sidang]> DELIMITER $$
MariaDB [db_sidang]>
MariaDB [db_sidang]> CREATE TRIGGER trg_log_sidang
->
-> AFTER INSERT
-> ON sidang
->
-> FOR EACH ROW
->
-> BEGIN
->
-> INSERT INTO log_activity(
-> aktivitas,
-> waktu
-> )
->
-> VALUES(
-> CONCAT('Tambah sidang mahasiswa ID ',NEW.id_mahasiswa),
-> NOW()
-> );
->
-> END $$
Query OK, 0 rows affected (0.021 sec)
```

3.7 Query Laporan

Query laporan digunakan untuk mengetahui mahasiswa yang belum memiliki jadwal sidang.

Gambar 3.7 Query Laporan

```
MariaDB [db_sidang]> SELECT *
-> FROM mahasiswa
-> WHERE id_mahasiswa NOT IN
-> (
-> SELECT id_mahasiswa
-> FROM sidang
-> );
+-----+-----+-----+-----+
| id_mahasiswa | nim    | nama | prodi |
+-----+-----+-----+-----+
| 3 | 2301003 | Siti | Sistem Informasi |
+-----+-----+-----+-----+
1 row in set (0.018 sec)

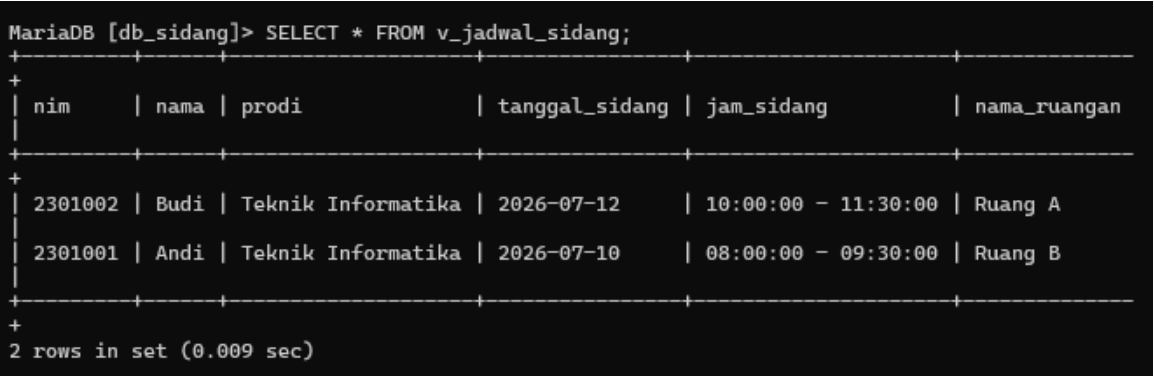
MariaDB [db_sidang]> |
```

BAB IV
HASIL DAN PEMBAHASAN
4.1 Hasil View

Setelah View dibuat, dilakukan pengujian menggunakan perintah:

SELECT * FROM v_jadwal_sidang;

Gambar 4.1 Hasil View



```
MariaDB [db_sidang]> SELECT * FROM v_jadwal_sidang;
```

nim	nama	prodi	tanggal_sidang	jam_sidang	nama_ruangan
2301002	Budi	Teknik Informatika	2026-07-12	10:00:00 - 11:30:00	Ruang A
2301001	Andi	Teknik Informatika	2026-07-10	08:00:00 - 09:30:00	Ruang B

2 rows in set (0.009 sec)

View berhasil menampilkan NIM, nama mahasiswa, program studi, tanggal sidang, jam sidang, dan ruangan sesuai data yang terdapat pada tabel.

4.2 Hasil Stored Procedure

Stored Procedure diuji menggunakan perintah:

```
CALL sp_tambah_sidang(1,'2026-07-10','08:00:00','09:30:00',2);
```

Gambar 4.2 Hasil Stored Procedure

```
MariaDB [db_sidang]> DELIMITER $$
MariaDB [db_sidang]>
MariaDB [db_sidang]> CREATE PROCEDURE sp_tambah_sidang(
  -> IN p_id_mahasiswa INT,
  -> IN p_tanggal DATE,
  -> IN p_jam_mulai TIME,
  -> IN p_jam_selesai TIME,
  -> IN p_id_ruangan INT
  -> )
  ->
  -> BEGIN
  ->
  -> INSERT INTO sidang(
  -> id_mahasiswa,
  -> tanggal_sidang,
  -> jam_mulai,
  -> jam_selesai,
  -> id_ruangan
  -> )
  ->
  -> VALUES(
  -> p_id_mahasiswa,
  -> p_tanggal,
  -> p_jam_mulai,
  -> p_jam_selesai,
  -> p_id_ruangan
  -> );
  ->
  -> END $$
Query OK, 0 rows affected (0.098 sec)

MariaDB [db_sidang]>
MariaDB [db_sidang]> DELIMITER ;
MariaDB [db_sidang]> CALL sp_tambah_sidang(
  -> 1,
  -> '2026-07-10',
  -> '08:00:00',
  -> '09:30:00',
  -> 2
  -> );
Query OK, 1 row affected (0.023 sec)
```

Data sidang berhasil ditambahkan ke dalam tabel sidang sesuai parameter yang diberikan.

4.3 Hasil Trigger

Trigger diuji dengan menambahkan data sidang melalui Stored Procedure.

Gambar 4.3 Hasil Trigger

```
MariaDB [db_sidang]>
MariaDB [db_sidang]> DELIMITER ;
MariaDB [db_sidang]> CALL sp_tambah_sidang(
    -> 2,
    -> '2026-07-12',
    -> '10:00:00',
    -> '11:30:00',
    -> 1
    -> );
Query OK, 2 rows affected (0.015 sec)

MariaDB [db_sidang]> SELECT * FROM log_activity;
+-----+-----+-----+
| id_log | aktivitas | waktu |
+-----+-----+-----+
| 1 | Tambah sidang mahasiswa ID 2 | 2026-07-09 00:50:54 |
+-----+-----+-----+
1 row in set (0.001 sec)
```

Hasil pengujian menunjukkan bahwa setiap penambahan data sidang akan secara otomatis menghasilkan data baru pada tabel log_activity.

4.4 Hasil Query Laporan

Query laporan dijalankan untuk mengetahui mahasiswa yang belum memiliki jadwal sidang.

```
SELECT *
FROM mahasiswa
WHERE id_mahasiswa NOT IN
(
SELECT id_mahasiswa
FROM sidang
);
```

Gambar 4.4 Hasil Query Laporan

```
MariaDB [db_sidang]> SELECT *
    -> FROM mahasiswa
    -> WHERE id_mahasiswa NOT IN
    -> (
    -> SELECT id_mahasiswa
    -> FROM sidang
    -> );
+-----+-----+-----+-----+
| id_mahasiswa | nim | nama | prodi |
+-----+-----+-----+-----+
| 3 | 2301003 | Siti | Sistem Informasi |
+-----+-----+-----+-----+
1 row in set (0.018 sec)

MariaDB [db_sidang]> |
```

Query berhasil menampilkan daftar mahasiswa yang belum memiliki jadwal sidang.

4.5 Analisis Hasil

Berdasarkan hasil pengujian, seluruh objek database berhasil dijalankan dengan baik. View mampu menyajikan data dari beberapa tabel menjadi satu tampilan. Stored Procedure mempermudah proses penambahan data karena cukup dipanggil menggunakan perintah CALL. Trigger berjalan secara otomatis untuk mencatat aktivitas penambahan data pada tabel log_activity. Query laporan juga berhasil menampilkan informasi mahasiswa yang belum memiliki jadwal sidang sesuai kebutuhan pengguna.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan dapat disimpulkan bahwa View, Stored Procedure, Trigger, dan Query Laporan dapat diterapkan dengan baik pada Database Sistem Sidang Tugas Akhir. Penggunaan fitur-fitur tersebut membantu mempermudah pengelolaan data, meningkatkan efisiensi kerja, serta menghasilkan informasi yang lebih cepat dan akurat.

5.2 Saran

Sebaiknya dilakukan pengembangan lebih lanjut pada Stored Procedure agar dapat mendeteksi bentrok jadwal sidang serta penambahan Trigger untuk proses UPDATE dan DELETE sehingga sistem menjadi lebih lengkap.

DAFTAR PUSTAKA

1. Modul Praktikum Aplikasi Basis Data Pertemuan 13, "Stored Procedure, Trigger, dan View pada Database Sistem Sidang Tugas Akhir".
2. Oracle. *MySQL 8.0 Reference Manual*.
3. MariaDB Documentation. *Stored Procedures, Triggers, and Views*.

LAMPIRAN

- Lampiran 1 : Screenshot pembuatan database.
- Lampiran 2 : Screenshot pembuatan tabel.
- Lampiran 3 : Screenshot input data.
- Lampiran 4 : Screenshot pembuatan View.
- Lampiran 5 : Screenshot pembuatan Stored Procedure.
- Lampiran 6 : Screenshot pembuatan Trigger.
- Lampiran 7 : Screenshot hasil View.
- Lampiran 8 : Screenshot hasil Stored Procedure.
- Lampiran 9 : Screenshot hasil Trigger.
- Lampiran 10 : Screenshot hasil Query Laporan.